

Este examen suma un total de 30 puntos. Cada 3 preguntas de test con 4 opciones o menos que se respondan de forma incorrecta se resta 1 punto. No está permitido el uso de calculadora. La duración del examen es de 70 minutos. Siga las instrucciones de la hoja de respuestas.

- 1 [1p] ¿Qué ventaja principal ofrece asyncio para implementar servidores?
 - ☐ a) Permite evitar por completo el uso de sockets.
 - ☐ b) Obliga a que cada cliente tenga un proceso dedicado.
 - ☒ c) Gestiona muchas conexiones concurrentes con un único bucle de eventos.
 - ☐ d) Convierte automáticamente el código TCP en UDP si hay demasiados clientes.
- 2 [1p] Un flujo UDP se identifica por...
 - ☐ a) Una tupla (ip, puerto).
 - ☒ b) dos tuplas (ip, puerto).
 - ☐ c) cuatro sockets.
 - ☐ d) Otra cosa.
- 3 [1p] ¿De qué nivel OSI son los sockets raw socket(AF_INET, SOCK_RAW, ...)?
 - ☐ a) Todos los sockets son nivel VII de aplicación, por eso se programan en lenguajes de alto nivel que importan clases basadas en POSIX.
 - ☐ b) Todos los sockets son conceptualmente nivel IV, no hay excepciones porque operan a nivel de transporte.
 - ☐ c) Todos los sockets son nivel III y por tanto también lo son los sockets raw.
 - ☒ d) Los sockets raw son una excepción ya que no usan puertos, por tanto operan directamente a nivel III incluso II.
- 4 [1p] ¿Qué caracteriza a un servidor multihilo de tipo *thread-per-connection*?
 - ☐ a) Usa un único hilo y delega todo en el sistema operativo.
 - ☒ b) Crea o asigna un hilo para atender cada conexión aceptada.
 - ☐ c) Comparte el mismo descriptor entre clientes, sin necesidad de accept().
 - ☐ d) Atiende una única conexión cada vez, pero con varias llamadas a recv().
- 5 [1p] ¿Cuál de las siguientes funciones del API de sockets convierte un socket TCP activo no conectado en un socket pasivo?
 - ☐ a) connect
 - ☐ b) recv
 - ☒ c) listen
 - ☐ d) accept
- 6 [1p] Dada la siguiente sentencia Python, indica qué opción es verdadera.

```
1 data = sock.recv(4096)
```

- ☐ a) El proceso se bloquea hasta recibir 4096 bytes.
- ☒ b) El proceso se bloquea hasta recibir al menos 1 byte hasta un máximo de 4096.
- ☐ c) El proceso almacenará consecutivamente 1024 bytes en cada lectura, bloqueando al principio de cada bloque de 1024.
- ☐ d) El método recv() nunca es un proceso bloqueante.

- 7 [1p] Considere el siguiente fragmento de código para un servidor **TCP secuencial**:

```
1 sock = socket(AF_INET, SOCK_STREAM)
2 sock.bind(('', 80))
3 sock.listen(51)
```

¿Cuántas peticiones de conexión pueden encolarse antes de que puedan ser aceptadas por este servidor?

- ☐ a) 1
- ☐ b) 5
- ☒ c) 51
- ☐ d) 50

- 8 [1p] El siguiente listado, correspondiente a un servidor TCP básico, contiene un error. ¿En qué línea?

```
1 sock = socket(socket.AF_INET, socket.SOCK_STREAM)
2 sock.connect(('', int(sys.argv[1])))
3 sock.listen(5)
4
5 while 1:
6     child_sock, client = sock.accept()
7     handle(child_sock)
```

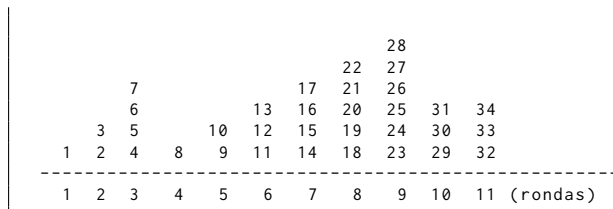
- ☐ a) línea 1.
- ☒ b) línea 2.
- ☐ c) línea 3.
- ☐ d) línea 6.

- 9** [1p] SSL/TSL provee a los sockets:
- ☐ a) Autenticación.
 - ☒ b) Autenticación y encriptación.
 - ☐ c) Autenticación, encriptación y tunelización.
 - ☐ d) Ninguna de las tres opciones.
- 10** [1p] Una conexión TCP siempre es iniciada...
- ☐ a) con un ISN igual a cero.
 - ☒ b) con el «triple apretón de manos».
 - ☐ c) por el servidor.
 - ☐ d) con $cwnd = 12 MSS$.
- 11** [1p] En TCP, ¿qué tienen en común los flags SYN y FIN respecto al número de secuencia?
- ☐ a) Ambos reinician el número de secuencia a 0.
 - ☐ b) Ninguno de los dos afecta al número de secuencia.
 - ☒ c) Ambos *consumen* una unidad del espacio del número de secuencia, aunque no transporten datos.
 - ☐ d) Ambos incrementan el número de secuencia en el tamaño de la carga útil del segmento.
- 12** [1p] ¿Qué determina el MSS en una conexión TCP?
- ☐ a) El tamaño máximo de la cabecera TCP, incluyendo opciones.
 - ☒ b) El tamaño máximo de la carga útil de los segmentos durante la conexión.
 - ☐ c) El tamaño máximo del datagrama IP, incluyendo cabeceras.
 - ☐ d) El tamaño del buffer de recepción del receptor.
- 13** [1p] ¿Cómo anuncia cada extremo TCP su MSS?
- ☒ a) Incluyendo la opción MSS en el segmento SYN.
 - ☐ b) En cualquier segmento con el flag ACK activo.
 - ☐ c) Mediante un segmento de control dedicado antes de enviar datos.
 - ☐ d) El MSS se calcula automáticamente a partir del campo window y no necesita anunciarse.
- 14** [1p] ¿En qué situación la desconexión TCP requiere 4 segmentos en lugar de 3?
- ☐ a) Cuando hay segmentos de datos pendientes de confirmación al inicio de la desconexión.
 - ☒ b) Cuando el receptor del FIN todavía tiene datos pendientes de enviar.
 - ☐ c) Cuando la desconexión la inicia el servidor en lugar del cliente.
 - ☐ d) Cuando expira el RTO durante el proceso de desconexión.
- 15** [1p] ¿Qué problema previene el estado TIME_WAIT (tiempo de silencio) en TCP?
- ☐ a) Que un servidor acepte conexiones antes de haber completado su inicialización.
 - ☐ b) Que dos conexiones simultáneas utilicen el mismo puerto de origen.
 - ☒ c) Que los segmentos de una conexión anterior sean confundidos con los de una nueva conexión.
 - ☐ d) Que el último ACK de la desconexión se pierda sin posibilidad de retransmisión.
- 16** [1p] ¿Cuál es el principal uso del flag RST en TCP?
- ☐ a) Indicar que el receptor no puede procesar el segmento por falta de espacio en el buffer.
 - ☐ b) Solicitar al emisor que reduzca el tamaño de la ventana de envío.
 - ☐ c) Notificar al emisor que el checksum del segmento recibido es incorrecto.
 - ☒ d) Rechazar una conexión entrante cuando el puerto está cerrado o no hay recursos para aceptarla.
- 17** [1p] ¿Qué indica el campo *window* en la cabecera TCP?
- ☐ a) El número de segmentos que se pueden enviar sin esperar confirmación.
 - ☐ b) El tamaño máximo de segmento negociado durante la conexión.
 - ☒ c) El espacio libre en el buffer de recepción del emisor de ese segmento.
 - ☐ d) El tamaño del buffer de envío del emisor de ese segmento.

18 [1p] ¿Qué hace el algoritmo de Nagle para paliar el síndrome de la *ventana tonta*?

- ☒ a) Si tiene datos sin confirmar, retiene los nuevos hasta confirmar los pendientes o acumular MSS bytes.
- ☐ b) Obliga al receptor a anunciar una ventana mínima igual al MSS antes de aceptar datos.
- ☐ c) Descarta segmentos con carga útil inferior a un *ssthresh*.
- ☐ d) Divide segmentos grandes en fragmentos para adaptarse al ancho de banda disponible.

A [5p] Considere el siguiente gráfico que representa la ventana de congestión de una conexión TCP. Los números indican el orden en que se envían los segmentos, con independencia de si son retransmisiones o no. Asuma que inicialmente *ssthresh* = $8MSS$ y siempre *rwnd* > *cwnd*. Responda a las siguientes preguntas:



> **19** (1p) Indique las rondas en las que se está ejecutando el algoritmo de Slow Start:

- ☒ a) 1-5
- ☐ b) 1-2 y 4-5.
- ☐ c) 1-2 y 10.
- ☐ d) 1-9

> **20** (1p) Indique al comienzo de qué ronda(s) cambia el valor de *ssthresh* y qué valor toma:

- ☐ a) Ronda 2, *ssthresh* = $3MSS$.
- ☒ b) Ronda 4, *ssthresh* = $2MSS$.
- ☐ c) Ronda 4, *ssthresh* = $2MSS$ y Ronda 10, *ssthresh* = $4MSS$.
- ☐ d) Ronda 4, *ssthresh* = $2MSS$; Ronda 10, *ssthresh* = $3MSS$ y Ronda 11, *ssthresh* = $2MSS$.

> **21** (1p) Indique durante qué ronda se reciben 3 ACKs duplicados

- ☒ a) 9
- ☐ b) 9 y 10.
- ☐ c) 4, 9 y 10.
- ☐ d) 4 y 10.

> **22** (2p) ¿En cuántas rondas se habrían enviado todos los segmentos si en lugar de producirse un RTO se hubieran recibido 3 ACK en esa ronda?

- ☐ a) 8 rondas
- ☒ b) 9 rondas
- ☐ c) 10 rondas
- ☐ d) 11 rondas, pero sólo quedaría un segmento por enviar.

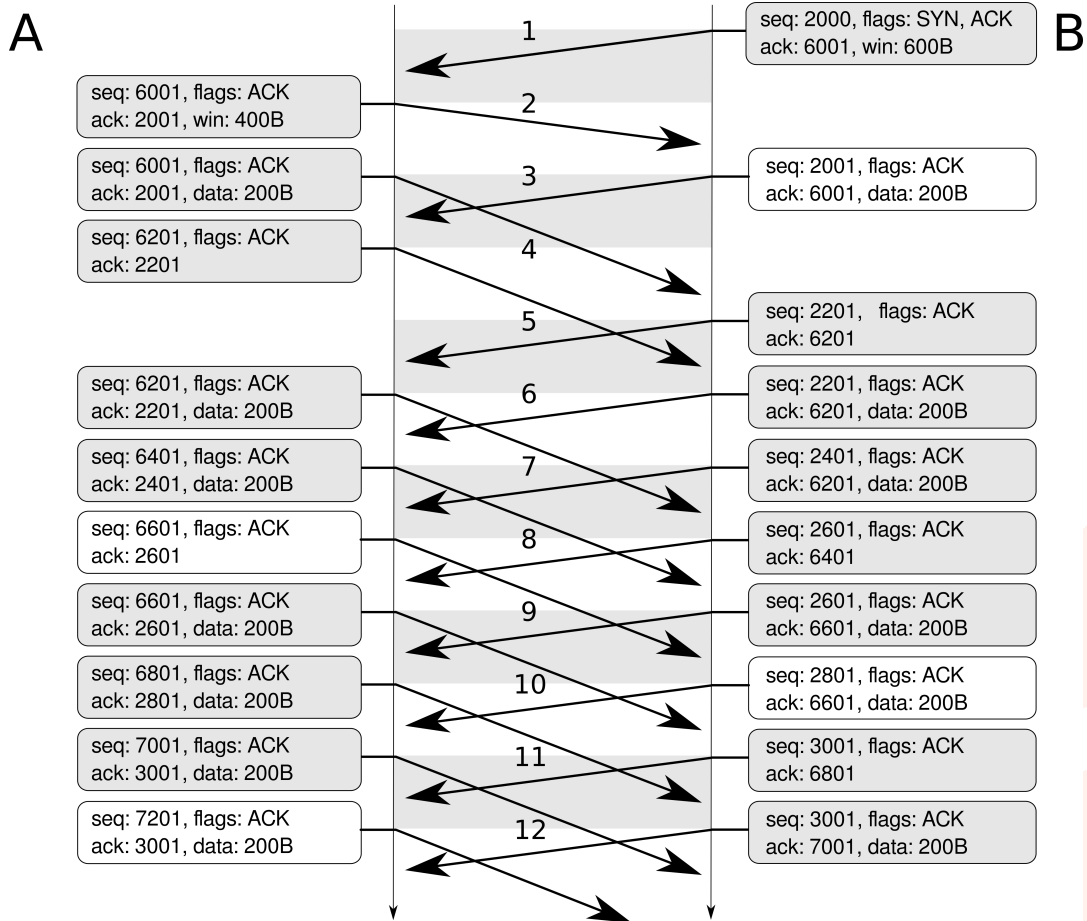
23 [1p] ¿Qué ocurre con el valor del RTO cada vez que expira sin recibir confirmación?

- ☐ a) Se divide entre dos para aumentar la frecuencia de retransmisiones.
- ☐ b) Se incrementa en un valor fijo equivalente al RTT medio medido.
- ☐ c) Se resetea al valor mínimo de 1 segundo para retransmitir lo más pronto posible.
- ☒ d) Se duplica (backoff exponencial) hasta un valor máximo de 60 segundos.

24 [1p] ¿Qué ventaja principal aporta SACK frente a la confirmación acumulativa convencional de TCP?

- ☐ a) Permite al receptor descartar segmentos fuera de orden sin almacenarlos en el buffer.
- ☐ b) Garantiza que las confirmaciones lleguen antes que los datos que reconocen.
- ☒ c) El emisor sabe qué segmentos fuera de orden llegaron correctamente y solo retransmite los que faltan.
- ☐ d) Elimina la necesidad del RTX para segmentos ya enviados.

- B** [5p] La figura muestra un flujo TCP incompleto que emplea control de congestión. Tenga en cuenta que la figura no incluye el primer segmento ni la parte final. Solo se envían segmentos coincidiendo con los ticks, MSS es 200 bytes, el plazo de retransmisión es 4 ticks, envían datos siempre que pueden y ambos deben enviar un total de 2000 bytes. Responda a las siguientes preguntas:



- > **25** (1p) ¿Qué debería enviar B en el tick 3?
- ☒ a) seq:2001, flags:ACK, ack:6001, payload:200B.
- ☐ b) seq:2001, flags:ACK, ack:6001.
- ☐ c) seq:2201, flags:ACK, ack:6001, payload:200B.
- ☐ d) seq:2201, flags:ACK, ack:6201, payload:200B.
- > **26** (1p) ¿Qué debería enviar A en el tick 8?
- ☐ a) seq:6601, flags:ACK, ack:2601, payload:200B.
- ☒ b) seq:6601, flags:ACK, ack:2601.
- ☐ c) seq:6401, flags:ACK, ack:2601, payload:200B.
- ☐ d) seq:6401, flags:ACK, ack:2601.
- > **27** (1p) ¿Qué debería enviar B en el tick 10?
- ☐ a) seq:2601, flags:ACK, ack:6601.
- ☐ b) seq:2601, flags:ACK, ack:6401, payload:200B.
- ☒ c) seq:2801, flags:ACK, ack:6601, payload:200B.
- > **28** (1p) ¿Qué debería enviar A en el tick 12?
- ☒ a) seq:7201, flags:ACK, ack:3001, payload:200B.
- ☐ b) seq:7201, flags:ACK,FIN, ack:3001.
- ☐ c) seq:7001, flags:ACK, ack:3001, payload:200B.
- ☐ d) seq:7001, flags:ACK,FIN, ack:3001.
- > **29** (1p) En relación a B ¿cuál es el valor de *cwnd*, *swnd* y el nivel de ocupación de dicha ventana antes de enviar el mensaje en el tick 12?
- ☐ a) *cwnd*=400B, *swnd*=400B, ocupación:100 %.
- ☒ b) *cwnd*=1000B, *swnd*=200B, ocupación:50 %.
- ☐ c) *cwnd*=400B, *swnd*=400B, ocupación:50 %.
- ☐ d) *cwnd*=400B, *swnd*=800B, ocupación:100 %.